# TAF course (*Preliminary*)

# Development of a pythonic Template of Analysis Framework (TAF)

◇ Git & Github project creation ◇ PYTHON tips & tricks ◇
◇ Analysis workflow development ◇
◇ Worlflow configuration ◇

TAF course (*Preliminary*)- November 3, 2026

**General notations of the document**

**Definition**

**Rule or convention**

**Method**

**Ex. 0.1 | Exercise**

**Nota bene**

Remark

Achtung!

Bonus

A few disclaimers :

- The pieces of advice given in this course are not universal nor unique, as there are often many possible approaches to a problem.
- This course is adapted to Unix and Mac operating systems. On Windows computer, one would have to work with a Unix kernel (as WSL).
- This is not a PYTHON course per say but several PYTHON-dedicated tips & tricks are scattered along the course.

## Prerequisites

- ▶ Programming :
  - ▷ some basic git knowledge
    - 💬 Not mandatory but preferable as the git functionalities used will not be presented in detail.
  - ▷ some basic PYTHON knowledge
    - 💬 PYTHON console, execution of a PYTHON script, variables, functions, ...
  - ▷ some basic Bash knowledge
    - 💬 Command lines in terminal, environment variables, ...
- ▶ Tools that must already be installed/prepared :
  - ▷ git
    - 💬 Should be natively available on all machines.
  - ▷ a github account
    - 💬 Ideally with SSH key configured, but not mandatory.
  - ▷ Visual Studio Code
    - 💬 Not mandatory but preferable as there will be dedicated exercises.
  - ▷ miniconda
    - 💬 Not mandatory but preferable as there will be dedicated exercises.

⚠ We will not be able do perform some "*installation customer service*" during this course given the time constraints.

TAF course (*Preliminary*)- November 3, 2026

Let's assume the following scenario :

You are a PhD student that just started the PhD. Your supervisor told you that you are going to pursue the work of a former PhD student, and that this student left his/her codes in a folder on a disc. Great! You think that you are lucky and that your analysis will quickly be on track... until you open the folder left by the former student : the files are not organised, the codes are not documented, there are no indications on how to use them, and the path to the files are hardcoded and are the absolute paths only valid in the machine of the former PhD student. And, even after struggling to understand how to run one of those codes, you realise that you are lacking some libraries for the codes to run!



The goal of this course is to give you the tools to build an Analysis Framework with files organisation and versioning, indications on how to read and run the codes, configurable workflows, ... in a few words : the complete opposite of the situation described above. This way, your colleagues or the next PhD student will not face such struggles.

# Course layout


**Chapter 1 - Creation of the project** **5**

I) Set up version control . . . 5
- I.1) Creation of a github repository . . . 5
- I.2) Local creation of a git project . . . 6
- I.3) Create a `dev` branch . . . 6
- I.4) Open a Pull Request . . . 8

II) Set up a virtual environment . . . 9
- II.1) Motivations . . . 9
- II.2) Using `conda` . . . 10
  - II.2.a) Write a configuration file . . . 10
  - II.2.b) The environment creation in itself . . . 11
- II.3) Using PYTHON `venv` . . . 12
  - II.3.a) Write a configuration file . . . 12
  - II.3.b) The environment creation in itself . . . 12

III) Installation guide in `README.md` . . . 13
- III.1) For the virtual environment creation . . . 13
- III.2) Create an environment variable for the main repository . . . 14
  - III.2.a) Creation and use . . . 14
  - III.2.b) Update the installation guide . . . 16

**Chapter 2 - PYTHON tips & tricks** **17**

I) Set up Visual Studio Code for PYTHON . . . 18

II) Some PYTHON tips for aesthetics (but not only...) . . . 19
- II.1) The `if __name__ == "__main__"` . . . 19
- II.2) The C-like architecture . . . 22
- II.3) Type hint . . . 23
- II.4) Comments & docstrings . . . 24
  - II.4.a) Docstring for a function or a method . . . 24
  - II.4.b) Docstring for a whole file . . . 25
  - II.4.c) Highlighted comments . . . 25
- II.5) Add flags to the `main` function . . . 26
- II.6) LOG messages . . . 27
  - II.6.a) The `Utils` folder . . . 27
  - II.6.b) Create a logger module . . . 27
  - II.6.c) Update `PYTHONPATH` . . . 27
  - II.6.d) Update installation guide in `README.md` . . . 29
- II.7) Create a Visual Studio Code snippet . . . 29

III) Some PYTHON tips for the actual development . . . 31
- III.1) `f-string` . . . 31


TAF course (*Preliminary*)- November 3, 2026

III.2) `list` . . . 31
III.3) `dict` . . . 33
III.4) Loop over iterables . . . 34

**Chapter 3 - Develop an analysis workflow** **37**
I) Import and read a `.root` file . . . 38
I.1) Open and read a `.root` file with `uproot` . . . 38
I.2) Read the file . . . 39
I.2.a) Get a `TTree` . . . 39
I.2.b) Convert a `TTree` in `pandas` dataframe . . . 39
I.3) The `Utils` folder . . . 40
I.3.a) *A parte* on code duplication . . . 40
I.3.b) Step 1 : create a helper function . . . 42
I.3.c) Step 2 : create a module . . . 42
I.3.d) Step 3 : call this module in a script . . . 42
II) *A parte* : make your script configurable . . . 43
II.1) Command line arguments . . . 43
II.2) YAML configuration files . . . 45
II.2.a) Set the `argparse` functionality . . . 45
II.2.b) Read a YAML file in PYTHON . . . 46
II.3) Make file reading configurable . . . 49
II.4) Add safeties . . . 49
III) Analyse the data... and save the results . . . 51
III.1) Data Analysis . . . 51
III.1.a) Some `pandas` tips . . . 51
III.1.b) Hands-on : energy deposits of photons and protons in a water volume . . . 54
III.2) Open and write a `.root` file with `uproot` . . . 56
III.2.a) Several writing options . . . 56
III.2.b) Write a dataframe as a `TTree` . . . 56
III.2.c) Update `taf_data_handler.py` module . . . 57
III.2.d) Hands-on : energy deposits of photons and protons in a water volume . . . 58
III.2.e) Make file-writing configurable . . . 58
IV) Plots production . . . 59

TAF course (*Preliminary*)- November 3, 2026

# Chapter 1

# Creation of the project

## Chapter outline


I) Set up version control . . . . . **5**
- I.1) Creation of a github repository . . . . . 5
- I.2) Local creation of a git project . . . . . 6
- I.3) Create a `dev` branch . . . . . 6
- I.4) Open a Pull Request . . . . . 8

II) Set up a virtual environment . . . . . **9**
- II.1) Motivations . . . . . 9
- II.2) Using `conda` . . . . . 10
  - II.2.a) Write a configuration file . . . . . 10
  - II.2.b) The environment creation in itself . . . . . 11
- II.3) Using PYTHON `venv` . . . . . 12
  - II.3.a) Write a configuration file . . . . . 12
  - II.3.b) The environment creation in itself . . . . . 12

III) Installation guide in `README.md` . . . . . **13**
- III.1) For the virtual environment creation . . . . . 13
- III.2) Create an environment variable for the main repository . . . . . 14
  - III.2.a) Creation and use . . . . . 14
  - III.2.b) Update the installation guide . . . . . 16


TAF course (*Preliminary*)- November 3, 2026

# I) Set up version control

## I.1) Creation of a github repository

Here, we are going to follow this quickstart from the Github documentation.

### The `README.md` file

The presence of such a file is very important. Ideally, there should be a `README.md` file in every directory within your repository.

> 💬 The `md` extension means "markdown". You can find many pieces of information about this online, but to get you started quickly, you could have a look at this cheat sheet.

## I.2) Local creation of a git project

> 💬 There are several ways of doing it. Here we will create a clone of the Github repository we just created.

⚠ When pushing your first commit, push it in the main branch. Do not create a new branch yet, we will do it later.

### Clone a Github repository

We will follow the procedure Cloning a repository available here.

> 💬 The cloning using SSH key would be the option to favour. However it requires SSH certificate authority : if you did not configure it, just go for the HTTPS option.

### Ex. 1.1 | Create a local version of the remote repository

Clone your Github repository at the desired location on your machine.

**Hint**

You could for instance place it in your `HOME` :

```
$ cd  # go home
$ # follow git clone procedure
```

Listing 1.1 – In the terminal

## I.3) Create a dev branch

### Develop on a branch dedicated to development

It can seem trivial, but you should not develop your project on the `main` branch, even if you are the only contributor.
You should always create a branch dedicated to development, that you could call `dev` for instance. Every local modification of the project should be done on this branch. Then, you would push this branch on the remote Github repo to finally operate a `Pull Request` to merge the `main` and `dev` branches.

TAF course (*Preliminary*)- November 3, 2026

> 💬 This is unavoidable in a case of a multi-contributor project.
> In the example of a group composed of Michael, Jim and Dwight, each one of them should create his own development branch :
>
> ```
> dev_Michael
> dev_Jim
> dev_Dwight
> ```

## Pull request

Pull requests are proposals to merge code changes into a project. A pull request is GitHub's foundational collaboration feature, letting you discuss and review changes before merging them. This helps teams work together, catch issues early, and maintain code quality. – Source

> If you want to open a Pull request but that there is still some work to do or that you want the insight of colleagues before making it official, and good option is to create a Draft Pull request.

> You can configure what are called Github Actions to automatise your Pull request and perform some checks before merging branches.

## Create a dev branch (locally and remotely)

All the following instructions should be done in the terminal.

1. Go to the project repository
2. To create a branch locally, type

   ```
   $ git checkout -b dev
   ```

   > 💬 You can of course change `dev` for the name of your choice.

   > 💬 The `checkout` command allows you to move to another branch, and the `-b` option allows you to create a new branch.
3. Get the name of the remote repo :

   ```
   $ git remote -v
   ```

   > 💬 We note it `REMOTE_REPO` here but you would need to change it by the actual result of the command above.
4. To also create this same branch remotely, push your newborn branch to `REMOTE_REPO` :

   ```
   $ git push -u REMOTE_REPO dev
   ```

   > 💬 The `-u` option in the `push` command is only needed for the first push.

TAF course (*Preliminary*)- November 3, 2026

**Ex. 1.2 | Create a dev branch**

Follow the method above to create your own branch dedicated to development, both locally and remotely.

At this point, you should have :

- a local Git repo with a `main` branch and a `dev` branch
- a remote Github repo with the same branches
- the `main` branch and a `dev` branch should be identical

⚠ From now on, and for the rest of this whole course, we will stay on the `dev` branch.

## I.4) Open a Pull Request

### Commit, push and open a Pull Request

#### Commit your modifications

1. Apply modifications to your project on the `dev` branch
2. Add your modifications :
   ```
   $ git add <put here the files you want to add>
   ```
3. Commit these changes :
   ```
   $ git commit -m "My commit for blablabla"
   ```

💬 This can be done numerous times without pushing.

#### Push your local version of the branch towards remote server

4. Push the new version of your branch to the remote repo :
   ```
   $ git push REMOTE_REPO dev
   ```

💬 This can be done numerous times without opening a Pull Request.

#### Open a Pull Request

5. Go to your Github repo, Github will automatically notice that the `dev` branch contains more commits than the `main` branch, and will propose you a Pull Request. You just have to follow the steps described on the web site.
   If you need more information, you can have a look at the documentation.

TAF course (*Preliminary*)- November 3, 2026

**Ex. 1.3 | Make your first pull request**

Update the `README.md` file by putting a title :

```
# Template of Analysis Framework (TAF)
```

Listing 1.2 – In `README.md`

Follow the procedure described above to :

1. make your first commit
2. push it
3. open your first Pull Request

> 💬 There are still many things that could be introduced concerning Git and Github. Here we shall stick to the basic tools presented above as we will not need more.
> If you seek more content on this topic, there is a very nice Software Carpentry Workshop that should answer many of your interrogations.

# II) Set up a virtual environment

## II.1) Motivations

**Use virtual environments**

When we work in a "real" environment of the computer and we wish to install new PYTHON libraries, some conflicts between libraries may occur (notably regarding required versions).

**Example**

On our computer, the library A is installed with version 1.5. For some reason, we need to install a new library (the library B). However, the library B requires a version of library A inferior or equal to 1.3 : conflict!
At this point, one could believe that one "simply needs to re-install library A with version 1.3".

HOWEVER!

A new player enters the game : the library C, that has been installed on the computer all along. And, bad luck, this library C requires a version of library A superior or equal to 1.5.

| Name of PYTHON library | Version | Necessity |
|---|---|---|
| A | 1.5 | / |
| B | / | A $\leqslant$ 1.3 |
| C | / | A $\geqslant$ 1.5 |

TAF course (*Preliminary*)- November 3, 2026

We are stuck! We could delete the library C to use libraries A and B, but this would only be a temporary solution. Indeed, the day where we will need this library C once again, we will need to re-install it and remember this versions conflict issue between libraries A, B and C.
Hence this solution is not viable, especially since we usually have more than three libraries installed on our computer, and these are as many potential conflicts.

The previous example highlights the necessity (and practicality) to use separate virtual environments.

> 💬 Coming back to the previous example, one could create two virtual environments :
>
> ▷ an environment where libraries A and B live in harmony (without library C);
>
> ▷ an environment where libraries A and C live in harmony (without library B).

## II.2) Using conda

### II.2.a) Write a configuration file

> 💬 There are many ways of creating and configuring a virtual environment in `conda`. Here we choose the most direct, fast and modular method : a YAML configuration file.

**YAML file**

A YAML file is recognisable by its extension `.yml`. It is a file format very often used for configuration files and that must respect a well-defined structure.

**Nota bene**

We will use YAML files as configuration files for PYTHON scripts (see Section YAML configuration files).

**YAML file to set up a virtual environment for TAF using conda**

**[Example]**

```yaml
name: taf_env
channels:
  - conda-forge
dependencies:
  - python=3.12
  - pyyaml
  - pandas
  - numpy
  - matplotlib
  - pip
  - pip:
    - argparse
```

Listing 1.3 – taf_env.yml

TAF course (*Preliminary*)- November 3, 2026

💬 The libraries under `dependencies` will be installed via `conda`. But if they are under `pip`, they will be installed via `pip`.

⚠ In this configuration file, the line

```
  - python=3.12
```

allows to require the desired version of PYTHON.

### II.2.b) The environment creation in itself

#### Create a virtual environment using conda

Once the preliminary work of creating the configuration file `taf_env.yml` is completed, one simply has to run a single command to create the virtual environment :

```
$ conda env create -f taf_env.yml
```

Listing 1.4 – In the terminal

#### Update an existing conda environment

If you wish to update your environment (*e.g.* add a library), you have to modify the YAML file and run :

```
$ conda env update --name taf_env --file taf_env.yml --prune
```

Listing 1.5 – In the terminal

#### Nota bene

One huge advantage of `conda`, that has no equivalent with PYTHON `venv` (see below), is the possibility to list all of the virtual environments created on the machine :

```
$ conda env list
```

Listing 1.6 – In the terminal

#### Enter/exit the environment

To enter the environment `taf_env` :

```
$ conda activate taf_env
```

Listing 1.7 – In the terminal

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

In your terminal, the information before the "$ " will indicate the name of the environment, like (it depends on the terminal) :

```
(taf_env) $
```

Listing 1.8 – In the terminal

To exit the environment :

```
(taf_env) $ conda deactivate
```

Listing 1.9 – In the terminal

## II.3) Using PYTHON venv

💬 The creation is a bit less direct : one needs to first create the environment and then install the libraries once inside.

### II.3.a) Write a configuration file

💬 venv does not use YAML files but simple ASCII `.txt` files.

**Text file to set up a virtual environment for TAF using venv [Example]**

```
pyyaml
pandas
numpy
matplotlib
argparse
```

Listing 1.10 – `requirements_py_venv_taf.txt`

💬 Note that the name of the environment is not specified here contrary to the `conda` method.

### II.3.b) The environment creation in itself

**Create a virtual environment using venv**

To create a virtual environment named `taf_env` with version 3.12 (this is an example) of PYTHON, run :

```
$ python3.12 -m venv taf_env
```

Listing 1.11 – In the terminal

⚠ You have to choose the right location on your computer, because the files associated to this virtual environment will be created at this location.

TAF course (*Preliminary*)- November 3, 2026

### Update an existing venv environment

1. Enter the environment :

```
$ source path/taf_env/bin/activate
```

Listing 1.12 – In the terminal

Of course, you will need to replace `path` by the location of your environment installation.

2. Run

```
$ pip install --upgrade pip
$ python3 -m pip install -r requirements_py_venv_taf.txt
```

Listing 1.13 – In the terminal

> 💬 Other tools exist (on top of conda and venv).

**Ex. 1.4 | Create a virtual environment dedicated to TAF**

Thanks to the methods described above, create a virtual environment dedicated to your Analysis Framework.

> 💬 You can use the tool you want between `conda` and `venv`.

# III) Installation guide in `README.md`

### Installation guide

It is a good habit to give some kind of installation guide in the `README.md` file.
This is useful for you, as it can help you keep track of the YAML file you used to create your virtual environment, but also to keep track of the evolution of this file along the project development (*e.g.* add libraries as the project gets bigger).

## III.1) For the virtual environment creation

Here is a summary version of the explanations given in the sections above. You could just add it to your `README.md` file.

```
# Installation

## Create a virtual environment

### Option 1 -- using ‘conda‘

To create the virtual environment configured in the file ‘taf_env.yml‘, one simply needs
    to run:
‘‘‘
conda env create -f taf_env.yml
‘‘‘
```

TAF course (Preliminary) - November 3, 2026

````
*Note: to update an existing virtual environment (if one wished to modify 'taf_env.yml'),
     run*
'''
conda env update --name taf_env --file taf_env.yml --prune
'''

### Option 2 -- using python 'venv'

To create a virtual environment named 'taf_env' with version 3.12 (this is an example) of
     Python, run:

'''
python3.12 -m venv taf_env
'''

*Careful: you have to choose the right location on your computer, because the files
    associated to this virtual environment will be created at this location.*

Once the environment is created, you can enter it via:

'''
source path/taf_env/bin/activate
'''

Once inside this environment, **and only once inside (!)**, run:
'''
pip install --upgrade pip
python3 -m pip install -r requirements_py_venv_taf.txt
'''

to install the requirements defined in the file 'requirements_py_venv_taf.txt'
````

Listing 1.14 – In `README.md`

**Ex. 1.5 | Guide for the virtual environment**

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

## III.2) Create an environment variable for the main repository

### III.2.a) Creation and use

**Make the path to the main repository generic**

When you are working locally on your machine, the path to the project repository (*i.e.* the so-called main repository) is not the same as the one on a colleague's machine. The solution is to create an environment variable for the main repository in the configuration file of the machine :

- ▷ `~/.bashrc` in Unix
- ▷ `~/.bash_profile` in MacOS

TAF course (*Preliminary*)- November 3, 2026

> 💬 In what follows, the configuration file will be referred as ~/.bash*
> It is up to you to choose the right name according to your operating system.

You should add the following line to this configuration file to create an environment variable named TAF :

```
export TAF="<put the path to your main repo>"
```

Listing 1.15 – In ~/.bash*

### Make the new environment variable active

After adding the environment variable

```
export TAF="<put the path to your main repo>"
```

Listing 1.16 – In ~/.bash*

there are two methods to activate it :

- ▷ Method 1 : simply open a new terminal.
  > 💬 This automatically executes the ~/.bash* file.
- ▷ Method 2 : execute the ~/.bash* file manually :

  ```
  source ~/.bash*
  ```

  Listing 1.17 – In the terminal

  > 💬 This has the advantage of staying in the same terminal.

The main idea is that once you followed either one of these two methods, the variable TAF will be define (as long as you keep the dedicated line in the ~/.bash* file).

### Use this variable

To access the value of the variable TAF, just type ${TAF}

**Example**

In the terminal, type

```
$ echo ${TAF}
```

to display the content of the variable TAF in the terminal, or

```
$ ls ${TAF}
```

to display the content of the main repository.

### Ex. 1.6 | Create an environment variable for the main repository

Use the methods above (and their examples) to define a new environment variable called TAF, and make it active in your terminal.

TAF course (*Preliminary*)- November 3, 2026

⚠ From now on, the location of the main repository in the computer will be accessed via `${TAF}`

### III.2.b) Update the installation guide

In the rest of the project, the path to the main repository will be replaced by the environment variable `TAF`, so it is absolutely crucial that every user defines this variable (otherwise, all the paths will be broken for the user!).
That's why you need to add pieces of information on how to define this variable and how to use it, inside the `README.md` in the `Installation` section :

```
## Create an environment variable for this repository

In your ‘~/.bashrc‘ (in Unix) or ‘~/.bash_profile‘ (in MacOS) add

‘‘‘
export TAF="<put the path to your local copy of this repo>"
‘‘‘

*Note: to get the local path to your repository, enter ‘pwd‘ in your terminal while being
    in your repo.*

*Note: the ‘export‘ keyword assumes that the shell is in BASH, it can vary if it is not (
   e.g. zsh, tcsh, ...).*

*Careful: do not forget to either open a new terminal or to ‘source‘ your ‘bash*‘ file to
    make this modification active.*

From now on, the location of this repo in the computer will be accessed via ‘${TAF}‘.
```

**Ex. 1.7 | Guide for the environment variable**

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

TAF course (*Preliminary*)- November 3, 2026

# Chapter 2

# PYTHON tips & tricks

## Chapter outline


I) Set up Visual Studio Code for PYTHON . . . . . **18**
II) Some PYTHON tips for aesthetics (but not only...) . . . . . **19**
II.1) The `if __name__ == "__main__"` . . . . . 19
II.2) The C-like architecture . . . . . 22
II.3) Type hint . . . . . 23
II.4) Comments & docstrings . . . . . 24
II.4.a) Docstring for a function or a method . . . . . 24
II.4.b) Docstring for a whole file . . . . . 25
II.4.c) Highlighted comments . . . . . 25
II.5) Add flags to the `main` function . . . . . 26
II.6) LOG messages . . . . . 27
II.6.a) The `Utils` folder . . . . . 27
II.6.b) Create a logger module . . . . . 27
II.6.c) Update `PYTHONPATH` . . . . . 27
II.6.d) Update installation guide in `README.md` . . . . . 29
II.7) Create a Visual Studio Code snippet . . . . . 29
III) Some PYTHON tips for the actual development . . . . . **31**
III.1) `f-string` . . . . . 31
III.2) `list` . . . . . 31
III.3) `dict` . . . . . 33
III.4) Loop over iterables . . . . . 34


A basic knowledge of PYTHON is required for this lecture. We will see features that you might already know and others that you do not (that will be of course of higher interest for you).

TAF course (*Preliminary*)- November 3, 2026

# I) Set up Visual Studio Code for PYTHON

## Open files or folders with Visual Studio Code

One option is to use the tools available in the `Explorer` of the Visual Studio Code window (top left).
Another option is to use the terminal :

```
$ code myFolder  # opens the whole folder named myFolder
# OR
$ code myFile  # opens the file named myFile
```

## Use Visual Studio Code terminal

In the Visual Studio Code window, among the tabs on the top left, you have a tab entitled `Terminal`. Just click it to open a terminal within the Visual Studio Code window.

> ⚠ Sometimes Visual Studio Code terminal can be a bit capricious and less robust than the native OS terminal. Note that it should not be the case with the tools used in this course.

## Useful extensions for PYTHON

In your Visual Studio Code window, click on the `Extensions` panel on the left tab. Then search and install the following extensions :

- `Python` : basic extension for PYTHON
- `Pylint` : to get hints on PYTHON writing conventions/guidelines
- `Black Formatter` : to auto format code
- `autoDocstring` : to automatically generate docstring (see Section Comments & docstrings)

**Ex. 2.1 | Install extensions**

Install the extensions mentioned above.

## Tune your extensions with a GUI



1. Go to the `Preferences` menu.
   > 💬 On MacOS, it is in the `Code` tab on the top left. On Unix, it is in the `File` tab.
2. Click on `Settings`
3. In the pop-up window, look for the `Extensions` entry and choose the Extension you want to tune.

TAF course (*Preliminary*)- November 3, 2026

### Tune your extensions via `settings.json`

The `settings.json` file is a configuration file. To open it, go to `View > Command Palette > Preferences: Open User Settings`

A window will pop up. It is a JSON file that can be written to set some settings.

**Example**

Here is an example of settings you could add to this file :

```
{
    "[python]": {
        "editor.defaultFormatter": "ms-python.black-formatter",
        "editor.formatOnSave": true,
    },
    "black-formatter.args": [
        "--line-length",
        "100"
    ],
    "autoDocstring.startOnNewLine": true,
    "files.autoSave": "afterDelay",
    "window.autoDetectColorScheme": true,
}
```

Listing 2.1 – `settings.json`

### Ex. 2.2 | Tune `Black Formatter` and `autoDocstring`

1. Copy-paste the example above in your `settings.json` file. This will tune `Black Formatter` and `autoDocstring` extensions, and enable autosave.
2. Test if `Black Formatter` works fine. To do so, go in any PYTHON script, put extra spaces here and there, then save the document.
   *It should autoformat itself.*

> We will test `autoDocstring` in the next section.

# II) Some PYTHON tips for aesthetics (but not only...)

## II.1) The `if __name__ == "__main__"`

On the web, in many forums, documentations, examples, ... you might find this `if __name__ == "__main__"` condition. What does it do? What is it used for?
In PYTHON, there are basically two types of `.py` files :

- the ones that are meant to be executed (what we call *scripts*);
- the ones that are meant to be imported in other `.py` files, and not meant to be executed (what we call *modules*).

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

The *script* or *module* status of a file is context-dependent. Indeed, one can have a `.py` file that is meant to be a *module* but that could be executed for debugging purposes (for instance).

In order to distinguish between these two types, there is something called the `__name__` variable.

**Ex. 2.3 | A repository for exercises**

In `${TAF}` repo, create a directory `Exercise`

**Hint**

You can directly use the "`New Folder...`" functionality in Visual Studio Code, or do it via the terminal :

```
$ mkdir ${TAF}/Exercise
```

Listing 2.2 – In the terminal

**Ex. 2.4 | The `__name__` variable**

1. Enter the `Exercise` directory
2. Create a directory `TestNameVariable`
3. Enter this directory
4. Create a file `script.py` such as :

```
print("In script.py, __name__: ", __name__)
```

Listing 2.3 – script.py

5. Create a file `module.py` such as :

```
print("In module.py, __name__: ", __name__)
```

Listing 2.4 – module.py

6. Execute the codes using :

```
$ python3 script.py
$ python3 module.py
```

Listing 2.5 – In the terminal

What is printed in the terminal ?

7. Now modify `script.py` by importing `module.py` :

```
import module
print("In script.py, __name__: ", __name__)
```

Listing 2.6 – script.py

TAF course (*Preliminary*)- November 3, 2026

8 Execute `script.py` :

```
$ python3 script.py
```

Listing 2.7 – In the terminal

What is printed in the terminal ? Is there a difference with the previous message ? If so, why ?

### The `if __name__ == "__main__"` condition

To ensure that a `.py` file will only be executed if it is the "main" one (*i.e.* the target of `python3` in the execution command line), one can add the

```
if __name__ == "__main__":
    # put code here
```

condition.

### Ex. 2.5 | Add the `if __name__ == "__main__"` condition

Add the `if __name__ == "__main__"` condition to the files `module.py` and `script.py`

### The `if __name__ == "__main__"` condition for a "main" script

In PYTHON, there is a convention : if your code is meant to be executed, you should embed it within the `if __name__ == "__main__"` condition.

> 💬 As a consequence, all the codes in scripts lie inside the
>
> ```
> if __name__ == "__main__":
>     # put code here
> ```
>
> block.
>
> 💬 We will see a "cleaner" way to proceed below.

**Example**

Let's modify `script.py` from the previous exercise as follows :

```
import module
if __name__ == "__main__":
    print("Hello from script.py")
```

Listing 2.8 – script.py

The execution gives this result :

```
$ python3 script.py
In module.py, __name__:  module
Hello from script.py
```

Listing 2.9 – In the terminal

TAF course (*Preliminary*)- November 3, 2026

The code from `script.py` is executed as it is the main code. However, the code of `module.py` is also executed while it has the status of *module*...

### The `if __name__ == "__main__"` condition for debugging/testing a module

An exception to the previous convention can happen in the following case : you have a module and want to perform direct debugging or testing. Then, you put this code inside a `if __name__ == "__main__"` condition so it does not get executed when the module is imported.

#### Example

Coming back to the example in the box above. If we want the code of `module.py` to be executed only if it is directly executed and not if it is imported, we should do :

```
if __name__ == "__main__":
    print("Hello from module.py")
    print("In module.py, __name__: ", __name__)
```

Listing 2.10 – module.py

When we want to debug/test the module, we can do :

```
$ python3 module.py
Hello from module.py
In module.py, __name__:  __main__
```

Listing 2.11 – In the terminal

and the code in `module.py` is executed.
However, if we want to use it as a *module* :

```
$ python3 script.py
Hello from script.py
```

Listing 2.12 – In the terminal

> 💬 Note that the code of `module.py` was not executed in this last command line.

## II.2) The C-like architecture

### The `main` function

We just saw that a PYTHON script should possess the `if __name__ == "__main__"` condition, and that its code should lie inside the condition block. This is not optimal, as it can alter the code readability.

To overcome this issue, we adopt an "architecture" inspired by the C language : a `main` function.

```
def main():
    print("This is my main function")
```

TAF course (*Preliminary*)- November 3, 2026

```python
if __name__ == "__main__":
    main()
```

Then, the "main" code will sit inside the `main` function and the `if __name__ == "__main__"` condition only embeds the call of the `main` function.

Why is it a good way to go ?

- ▷ in one quick look at the very end of the script, on can easily get a peak at the structure of the script ;
- ▷ if one wants a better understanding of the content of the script, one sees that there is a `main` function that is called, goes check what is inside this `main` function, that may itself call some helper functions, etc...

**Ex. 2.6 | `main` function convention**

In `script.py`, implement the `main` function convention.

## II.3) Type hint

### Type hint

In PYTHON, type hints allow developers to *annotate* their code with *expected types* for variables and function arguments and return. A type hint is written as follows :

```python
variable: type = value
```

> 💬 This can make the code more easily readable and also, possibly, allow to catch errors before runtime. The latter requires the use of external tools : so-called *type checkers*.

> 💬 This looks a lot like C for instance, except that in C, if you do not specify the type of your variable, the code will not compile, while in PYTHON it is just a hint : it will not lead to errors (if you do not apply some external type checker).

**Example**

```python
a: int = 1
pi: float = 3.14
name: str = "Jim"
names: list[str] = ["Michael", "Jim", "Dwight"]

def print_hello() -> None:
    print("Hello")

def sum(x: float, y: float) -> float:
    return x + y
```

TAF course (*Preliminary*)- November 3, 2026

**Ex. 2.7 | Type hinting**

In `script.py`, add a type hint to the `main` function.

## II.4) Comments & docstrings

In your VS Code window, you should see that `def main` is underlined (thanks to Pylint extension). If you hover your mouse over `def main`, a window will pop up with a message saying "Missing function or method docstring".

**docstring**

It is an element of code that will not be interpreted (it has the status of a comment) and that aims at providing pieces of information regarding a given function or the whole document.
The docstring environment is surrounded by triplets of " :

```
"""
Docstring here
"""
```

**docstring**

Once written, the docstring can be used to generate a documentation.

> This aspect will not be addressed in this course.

Also, in VS Code (for instance), if you hover the mouse over some function (and if its docstring is written), this gives you direct access to its documentation via a pop-up window.

### II.4.a) Docstring for a function or a method

**Use `autoDocstring`**

Type `"""` right after the `:` of a function definition. An auto-completion pop-up will appear : press `Tab` or `Enter` to run `autoDocstring`.

**Ex. 2.8 | Docstring the `main` function**

Use `autoDocstring` on the `main` function of `script.py`

**Ex. 2.9 | Docstring a function with arguments and a return**

In `script.py`, define a dummy function like :

```
def sum(a: float, b: float) -> float:
    return a + b
```

Use `autoDocstring` on the `sum` function.

TAF course (*Preliminary*)- November 3, 2026

### II.4.b) Docstring for a whole file

**Docstring of a file**

A `.py` file should start with a docstring like the following one :

```
"""
file: myscript.py
brief: Simple script to show how to define dosctrings
usage: python3 myscript.py
note:
author: Alexandre BIGOT, alexandre.bigot@iphc.cnrs.fr
"""
```

> 💬 This is an example, there are several ways to format such a "file docstring".

**Ex. 2.10 | Docstring a file**

Docstring the file `script.py`

### II.4.c) Highlighted comments

**Highlighted comments**

Comments containing a specific keyword that allows for them to be highlighted in your text editor (Visual Studio Code here).

**Nota bene**

The keywords used most often are `TODO` and `FIXME`.

**Example**

Let's suppose you are developing a code and do not have time to implement a given functionality at this very instant. In order to keep track of this in your code, you could put a comment like this one :

```
# TODO: add functionality blablabla
```

Also, if you are implementing a temporary solution or that you think a part of your code requires some debugging, you could add a comment like this one :

```
# FIXME: temporary solution
```

**Ex. 2.11 | Highlighted comment**

In `script.py`, add a `TODO` comment indicating that you will need to add a debug flag (see next section).

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

The docstring (and comments in general actually) might seem anecdotal at first sight (especially when you are developing your code, on the moment you just want to move forward and not lose time documenting the code...) but it is very important for one simple reason : you should not forget that you are developing code for your future self and/or other users!

- Your future self might be you in
  - 1 week : in this case, you should still have a good and direct understanding of the code because it is still fresh in your memory;
  - 1 month : you could have forgotten some details here and there but you could still catch up quite fast;
  - or 2 years : then it can become quite difficult to understand the code and what you had in mind at the time you wrote it.
- For other users, it is even harder because they are not in your head and it can be time-consuming to explain them the code (which would require you to master the code, looping back to the previous item).

To wrap up :

*At the moment you write the code, you should consume a little bit of time to write a clear documentation in order to save lots of time in the future ☺*

## II.5) Add flags to the `main` function

**Flag**

A flag is a boolean variable, a switch that may be on (`True`) or off (`False`).

**Flag in the `main` function**

It is a flag that is an argument of the `main` function.

**DEBUG flag**

It can be very useful (it is recommend to do it every time) to define a **DEBUG** flag, as follows :

```
def main(debug: bool) -> None:
    """
    Main function

    Parameters
    ----------------------------------------------
    - debug: bool
        Switch for debugging
    """
    if debug:
        print("DEBUG mode activated!")
```

TAF course (*Preliminary*)- November 3, 2026

```python
if __name__ == "__main__":
    DEBUG: bool = True
    main(DEBUG)
```

This way, you can put some debugging messages or checks in your `main` function via `if debug:` environments.

> 💬 Note that the `DEBUG` variable given as argument to the call of the `main` function is in capital letters as it is a constant (this is a PYTHON convention).

**Ex. 2.12 | DEBUG flag**

In `script.py`, add a `DEBUG` flag.

> 💬 Once this is done, you can remove the `TODO` comment added in Ex. 2.11.

## II.6) LOG messages

### II.6.a) The `Utils` folder

**`Utils` folder**

Generic name for a folder that shall welcome all relevant modules (for data handling, analysis purposes, plots production, ...) throughout the project.

**Ex. 2.13 | Creation of the `Utils` folder**

At the top level of your `TAF` project (*i.e.* in the `${TAF}` repo), create a folder named `Utils`

### II.6.b) Create a logger module

**Ex. 2.14 | Logger module**

Inside the `Utils` folder, download the PYTHON module named `taf_logger.py` available in the Seafile folder.

### II.6.c) Update `PYTHONPATH`

Now, if we want to import the `Logger` class from the `taf_logger.py` module, we must give the position of this module in our machine (so PYTHON can know its location and import it). To avoid this issue, one can modify the `PYTHONPATH` environment variable.

> 💬 This is *a* possible solution. There are others that can be quite complex.

TAF course (*Preliminary*)- November 3, 2026

### Add `Utils` folder to the path of known PYTHON modules

Add the following line in the ~/.bashrc (in Unix) or ~/.bash_profile (in MacOS) :

```
export PYTHONPATH="${PYTHONPATH}:${TAF}/Utils"
```

⚠ Do not forget to source your ~/.bash* file or to open a new terminal to make this modification active.

💬 Remember we already mentioned environment variables in Section Create an environment variable for the main repository.

### Ex. 2.15 | 

Add `Utils` folder to the path of known PYTHON modules.

### Ex. 2.16 | Test if this updated PYTHONPATH works

1. Enter the `${TAF}/Exercise` directory
2. Create a `Logger` directory and enter it
3. Create a file `test_logger.py`, and paste the following code in it :

```python
"""
file: test_logger.py
brief:
usage: python3 test_logger.py
note:
author: Alexandre BIGOT, alexandre.bigot@iphc.cnrs.fr
"""

from taf_logger import Logger as LOG


def main(debug: bool) -> None:
    """
    Main function

    Args:
        debug (bool): Switch for debugging
    """
    if debug:
        LOG("Debug mode activated!", "DEBUG")


if __name__ == "__main__":
    DEBUG: bool = True
    main(DEBUG)
```

4. Execute the script and check if it works fine.

TAF course (*Preliminary*)- November 3, 2026

**Ex. 2.17 | LOG messages**

We will add some content to `test_logger.py` :

1. a `RESULT` message
2. a `INFO` message
3. a `WARNING` message
4. a `ERROR` message
5. a `FATAL` message

What happens when you add a `FATAL` message ?

### II.6.d) Update installation guide in `README.md`

⚠ The previous modification on `PYTHONPATH` redefines the known paths for PYTHON `import` command. If a third-party user is not aware of this, the scripts will not work on his/her machine !
It is mandatory to indicate it in `README.md`

```
## Python path to 'Utils' modules

In order to be able to easily import TAF 'Utils' modules in local scripts, a solution is
    to add the following line to your '~/.bashrc' or '~/.bash_profile':
'''
export PYTHONPATH="${PYTHONPATH}:${TAF}/Utils"
'''

*Note: doing so, we will not need to specify the path of the module relative to the
    script we are developping.*

*Careful: do not forget to either open a new terminal or to 'source' your 'bash*' file to
     make this modification active.*
```

Listing 2.13 – In `README.md`

**Ex. 2.18 | Guide for the `PYTHONPATH` update**

1. Copy-paste the markdown code given above into your own `README.md`
2. Commit this modification thanks to `git commit`

## II.7) Create a Visual Studio Code snippet

**Add a Visual Studio Code snippet**

1. Open a Visual Studio Code window
2. Select `Configure Snippets` in `File > Preferences` (for Unix) or in `Code > Preferences` (for MacOS)
3. Select the language (PYTHON)

TAF course (*Preliminary*)- November 3, 2026

> 💬 This opens the file `python.json` containing the definitions of snippets.

### Ex. 2.19 | Add a snippet for PYTHON template and use it

1. Add the following lines between the main { } of the `python.json` file.

```
"TAF Python File Template": {
    "prefix": "py_tmpl_taf",
    "body": [
        "\"\"\"",
        "file: ${TM_FILENAME}",
        "brief:",
        "usage: python3 ${TM_FILENAME}",
        "note:",
        "author: Name, email address",
        "\"\"\"",
        "",
        "from taf_logger import Logger as LOG",
        "",
        "",
        "def main(debug: bool) -> None:",
        "    \"\"\"",
        "    Main function",
        "",
        "    Args:",
        "        debug (bool): Switch for debugging",
        "    \"\"\"",
        "    if debug:",
        "        LOG(\"Debug mode activated!\", \"DEBUG\")",
        "",
        "",
        "if __name__ == \"__main__\":",
        "    DEBUG: bool = True",
        "    main(DEBUG)",
        ""
    ],
    "description": "Template for new Python files in TAF"
}
```

Listing 2.14 – To put between the global { } envrionment

2. Create a new `.py` file with any name and open it in Visual Studio Code
3. Type `py_tmpl_taf`

> 💬 You should see some auto-completion proposition : press `Tab` or `Enter` to accept it and run the snippet.

### Ex. 2.20 | Commit and push the content of this Section

1. `git add` the `Exercise` and `Utils` folders

> 💬 No need to `add` the `README.md` as it was already added (and commited) in Ex. 2.18.

TAF course (*Preliminary*)- November 3, 2026

2. `git commit` these modifications
3. Push your `dev` branch

# III) Some PYTHON tips for the actual development

**Ex. 2.21 | Sandbox**

1. Enter the `${TAF}/Exercise` directory
2. Create a `PythonTips` directory
3. Among the tools introduced in this section, take some time to perform some tests (*e.g.* copy-paste the codes given as examples in dedicated scripts) whenever you encounter a new notion that you are not sure to fully understand.

## III.1) `f-string`

**`f-string`**

`f-strings` are string literals prefixed with `"f"` that contain expressions inside curly braces {}.

**Example**

```
pi = 3.141592
my_fstring = f"pi = {pi}"
print(my_fstring)
my_string_with_4decimals = f"pi = {pi:.4f}"
```

Listing 2.15 – `example.py`

```
$ python3 example.py
pi = 3.141592
pi = 3.1415
```

## III.2) `list`

⚠ PYTHON lists are one of the most important features you need to master.

**Construct a `list` object**

Use the `[]` notation. The comma `", "` is used to separate the elements within a list.

```
$ python3
>>> lempty = []
>>> print(lempty)
[]
```

TAF course (*Preliminary*)- November 3, 2026

```
>>> l1 = [1, 2, 3, 4]
>>> l2 = ["d", "w", "i", "g", "h", "t"]
>>> l3 = [None, 3.14, None]
>>> l1 + l2 +l3
[1, 2, 3, 4, 'd', 'w', 'i', 'g', 'h', 't', None, 3.14, None]
```

💬 The + operator is used to concatenate lists.

**Nota bene**

A PYTHON list can accept multiple types of variables.

⚠ `list` is the name of the constructor of the `list` class, so you cannot name a variable `list`!

## Create a list of size n

It can sometimes be better suited to work with a list of size `n` than with an empty list.

```
$ python3
>>> n = 5
>>> [False] * n
[False, False, False, False, False]
```

⚠ The list is filled with the same value for each entry.

## Use a PYTHON expression to create a list

Let's suppose you want to create a list with integers from 0 to 9. You could do

```
$ python3
>>> l = []
>>> for i in range(10):
...     l.append(i)
...
>>> print(l)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

But there is a one-liner way thanks to a PYTHON expression :

```
$ python3
>>> l = [i for i in range(10)]
>>> print(l)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

💬 There is also a way to do this thanks to `numpy` :

```
$ python3
>>> import numpy as np
>>> l = np.arange(0, 10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

But then `l` would be a numpy array and not a list.

TAF course (*Preliminary*)- November 3, 2026

### III.3) `dict`

⚠ A basic understanding of dictionaries is mandatory for the part of the lecture dedicated to the use of configuration files (see Section YAML configuration files).

## Dictionary (`dict` class)

Dictionary is a data structure that stores information in key-value pairs.

💬 It resembles the `map` in C++.

💬 In PYTHON, a dictionary is an object of type `dict`

To build a dictionary, you can use the `{ ... }` notation :

```
my_dict = {"key1": value1, "key2": value2}
```

⚠ `dict` is the name of the constructor of the `dict` class, so you cannot name a variable `dict`!

⚠ The *keys* must be unique and immutable (like strings or numbers).
The *values* may be of any data type, whether mutable or immutable.

**Nota bene**

This implies that a *value* can be a `list` or even a `dict`! It is not necessarily a `int`, `float` or a `string`.

💬 Dictionaries are especially suited for accessing data by a specific name rather than a numeric position like in `list`.

## Manipulate a `dict` object

We use the following example :

```
>>> salesman = {"name": "Dwight", "age": 30}
>>> salesman
{'name': 'Dwight', 'age': 30}
```

To retrieve the value of a given *key*, you need to put the key between `[ ]` after the dictionary name. For instance :

```
>>> salesman["name"]
'Dwight'
```

To add a new key-value pair, this is the exact same notation, except that you add a `= value`. For instance :

```
>>> salesman["rank"] = "Assistant (to the) regional manager"
>>> salesman["rank"]
'Assistant (to the) regional manager'
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Assistant (to the) regional manager'}
```

You can also set a new value for an existing key :

```
>>> salesman["rank"] = "Regional manager"
>>> salesman["rank"]
```

TAF course (*Preliminary*)- November 3, 2026

```
'Regional manager'
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Regional manager'}
```

**The `items` method**

To extract the key-value pairs, you can use :

```
key_value_pairs_items = my_dict.items()
```

**Nota bene**

This is a must when looping over a dictionary (see next section).

If you try to print it right as it is, you will not be able to manipulate it as a `list` :

```
>>> salesman.items()
dict_items([('name', 'Dwight'), ('age', 30), ('rank', 'Regional manager')])
>>> list(salesman.items())
[('name', 'Dwight'), ('age', 30), ('rank', 'Regional manager')]
```

## III.4) Loop over iterables

**Iterable**

An iterable corresponds to any PYTHON object capable of returning its members one at a time. As a consequence, it to be iterated over in a for-loop.

In the methods below, we will consider a list :

```
names = ["Michael", "Jim", "Dwight"]
```

**Loop over indices only**

We use the `range` method :

```
for iname in range(len(names)):
    print(f"Index: {iname}, element: {names[i]}")
```

- This requires to access the length of the list, using `len`
- This method gives access to both elements (indirectly) and indices.

**Nota bene**

The iteration index is here called `iname` and not `i` for instance. This may seem anecdotal but this is quite important : remember you should give meaning to your variables names!

TAF course (*Preliminary*)- November 3, 2026

## Loop over elements only

```
for name in names:
    print(f"Element: {name}")
```

> 💬 This is what we call a "for each" loop : for each iteration, we access the element of the list.

> 💬 This only gives access to the elements and not to their index in the list.

## Loop on both indices and elements
## The `enumerate` keyword

```
for iname, name in enumerate(names):
    print(f"Index: {iname}, element: {name}")
```

> 💬 This method gives direct access to both elements and indices.

## `enumerate` function

Function used to iterate over an *iterable*[a], returning pairs in the form of tuples shaped as `(index, element)`.

### Example

```
>>> names = ["Michael", "Jim", "Dwight"]
>>> print(f"Print enumerate output: {enumerate(names)}")
Print enumerate output: <enumerate object at 0x107943290>
>>> print(f"Print list of enumerate output: {list(enumerate(names))}")
Print list of enumerate output: [(0, 'Michael'), (1, 'Jim'), (2, 'Dwight')]
```

Listing 2.16 – In PYTHON console

> 🚀 One needs to convert enumerate output into a list otherwise we do not see what is within via print

a. It can be a `list` object or a `dict_items` object, for instance.

## Loop over a dictionary

As mentioned in the previous section, a good option is to use the `items` method.

### Example

Let's come back to the example of the previous section :

```
>>> salesman
{'name': 'Dwight', 'age': 30, 'rank': 'Regional manager'}
>>> for key, value in salesman.items():
...     print(f"key: {key}, value: {value}")
```

TAF course (*Preliminary*)- November 3, 2026

```
...
key: name, value: Dwight
key: age, value: 30
key: rank, value: Regional manager
```

### zip function

Used to combine multiple iterable objects (*e.g.* lists, tuples, or strings) into a single iterable of tuples.

#### Example

```
>>> names_men = ["Michael", "Jim", "Dwight"]
>>> names_women = ["Holly", "Pam", "Angela"]
>>> for name_man, name_woman in zip(names_men, names_women):
...     print(f"{name_man} and {name_woman} form a couple")
...
Michael and Holly form a couple
Jim and Pam form a couple
Dwight and Angela form a couple
```

Listing 2.17 – In PYTHON console

⚠ This example shows a `zip` on two iterables, but we could also do it on three or more iterables.

### `enumerate` and `zip` combo

As the output of `zip` is an iterable, one can use `enumerate` : to make it short, one can `enumerate` a `zip`!

#### Example

```
>>> names_men = ["Michael", "Jim", "Dwight"]
>>> names_women = ["Holly", "Pam", "Angela"]
>>> for icouple, (name_man, name_woman) in enumerate(zip(names_men, names_women)):
...     print(f"Couple {icouple}: {name_man} and {name_woman}")
...
Couple 0: Michael and Holly
Couple 1: Jim and Pam
Couple 2: Dwight and Angela
```

Listing 2.18 – In PYTHON console

TAF course (*Preliminary*)- November 3, 2026

# Chapter 3

# Develop an analysis workflow

## Chapter outline


I) Import and read a `.root` file . . . **38**
- I.1) Open and read a `.root` file with `uproot` . . . 38
- I.2) Read the file . . . 39
  - I.2.a) Get a `TTree` . . . 39
  - I.2.b) Convert a `TTree` in `pandas` dataframe . . . 39
- I.3) The `Utils` folder . . . 40
  - I.3.a) *A parte* on code duplication . . . 40
  - I.3.b) Step 1 : create a helper function . . . 42
  - I.3.c) Step 2 : create a module . . . 42
  - I.3.d) Step 3 : call this module in a script . . . 42

II) *A parte* : make your script configurable . . . **43**
- II.1) Command line arguments . . . 43
- II.2) YAML configuration files . . . 45
  - II.2.a) Set the `argparse` functionality . . . 45
  - II.2.b) Read a YAML file in PYTHON . . . 46
- II.3) Make file reading configurable . . . 49
- II.4) Add safeties . . . 49

III) Analyse the data... and save the results . . . **51**
- III.1) Data Analysis . . . 51
  - III.1.a) Some `pandas` tips . . . 51
  - III.1.b) Hands-on : energy deposits of photons and protons in a water volume . . . 54
- III.2) Open and write a `.root` file with `uproot` . . . 56
  - III.2.a) Several writing options . . . 56
  - III.2.b) Write a dataframe as a `TTree` . . . 56
  - III.2.c) Update `taf_data_handler.py` module . . . 57
  - III.2.d) Hands-on : energy deposits of photons and protons in a water volume . . . 58
  - III.2.e) Make file-writing configurable . . . 58


TAF course (*Preliminary*)- November 3, 2026

### Basic architecture of an analysis workflow

Generally, an analysis workflow is structured as follows :

1. import and read some data file
2. the actual data analysis (*e.g.* apply selections, build new observables from the ones already in the data)
3. save the analysed data in a file
4. produce some plots for the results

> 💬 This workflow is valid for any kind of data file. In this course, we will focus on `.root` files.
> However, the tools developed here for a `.root` file could be adapted to any other file.

> 💬 Of course, this is a basic version of a workflow. It would need to be adapted to your own context and needs. However, the tools given in this course are quite generic and could be re-used for a more advanced workflow.

# I) Import and read a `.root` file

For this section, we will need a script importing the following libraries :

```
import uproot
import pandas as pd
import numpy as np
```

### The `uproot` library

Uproot is a library for reading and writing ROOT files in pure PYTHON and NumPy. – Source

## I.1) Open and read a `.root` file with `uproot`

### The keyword `with`

It is a tool allowing for a clean handling of a file streaming (*i.e.* reading or writing) :

```
with open("sample.txt", "r") as file:
    # do something with file
    # file automatically closed at the end of the indented block
```

> 💬 This tool is quite handy as it automatically handles the closure of the file at the end of the `with` block.

TAF course (*Preliminary*)- November 3, 2026

### Open, read and close a file with uproot

To open a file names `name_infile`, we simply use the method `open` :

```
name_infile: str = "path_to_file.root"
with uproot.open(name_infile) as file:
    # put code here
```

Listing 3.1 – In the main function of `test_read_file.py`

> 💬 As mentioned above, the `with` keyword allows for an automatic handling of the file closure (at the end of the `with` block).

> ⚠ Note that here we use the `open` method of `uproot` to... open a file.
> To write a file, we will use other methods, as presented in Section Open and write a `.root` file with `uproot`.

## I.2) Read the file

### I.2.a) Get a TTree

### Get a TTree from a .root file with uproot

To get a `TTree` named `tree` inside the file `name_infile` :

```
name_infile: str = "path_to_file.root"
name_tree: str = "name of the TTree you want to get"
with uproot.open(name_infile) as file:
    tree: uproot.TTree = file[name_tree]
```

Listing 3.2 – In the main function of `test_read_file.py`

### I.2.b) Convert a TTree in pandas dataframe

### Direct conversion

You need to call the `arrays` method of the `uproot.TTree` class and set the `library` argument to `"pd"` :

```
name_infile: str = "path_to_file.root"
name_tree: str = "name of the TTree you want to get"
name_branches: str | list[str] | None = None
with uproot.open(name_infile) as file:
    # get the TTree
    tree: uproot.TTree = file[name_tree]
    # get pandas dataframe from TTree
    df: pd.DataFrame = tree.arrays(name_branches, library="pd")
```

Listing 3.3 – In the main function of `test_read_file.py`

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

The variable `name_branches` should be a list of strings (*i.e.* the list of `TBranch` objects you want to load within the `TTree` you are reading) or a string (if you want only one branch). If you want to get all the branches, juste set it to `None` (as done in this example).

**Indirect conversion (using `numpy`)**

If you are interested in manipulating `numpy` arrays rather than `pandas` dataframes or on top of `pandas` dataframes (for some reason), it is possible.
The indirect conversion consists in :

1. converting the `TTree` in a dictionary of `numpy` arrays
2. call the `pandas.DataFrame` constructor

```
name_infile: str = "path_to_file.root"
name_tree: str = "name of the TTree you want to get"
name_branches: str | list[str] | None = None
with uproot.open(name_infile) as file:
    # get the TTree
    tree: uproot.TTree = file[name_tree]
    # convert TTree in a dictionary of numpy arrays
    dict_np_arrays: dict[np.ndarray] = tree.arrays(name_branches, library="np")
    # construct a pandas dataframe
    df: pd.DataFrame = pd.DataFrame(dict_np_arrays)
```

Listing 3.4 – In the main function of `test_read_file.py`

> Note that the `tree` is converted in `numpy` via the `library="np"` argument in the `arrays` method.

## I.3) The `Utils` folder

### I.3.a) *A parte* on code duplication

Now that you know how to read a `.root` file and extract a `TTree` in the format of a `pandas` dataframe, you should see that the few lines of code only depend on the names of the input file, of the `TTree` and of the branches you want to extract. In other words, the code is quite generic and it would be a shame to leave it like this and copy-paste it every time we need to open a `.root` file.

**Say no to code duplication !**

While developing a project, you may realise that you re-define some *helper functions* in many scripts (*e.g.* a function to handle data files). This is a very bad habit.

TAF course (*Preliminary*)- November 3, 2026

**Example**

Let's assume you have a helper function you use to open a file and extract data from it, in your favourite format. You may use it at the beginning of the workflow (to get the original data) but also in your script to produce plots (at the end of the workflow). So, you have two copies of the same function in two different files.
Now, let's assume that, for some reason, you need to modify this function (*e.g.* make it more configurable or add a new feature). You would need to apply this modification to both copies of your function.
In this simple example, it may seem anecdotal but it is an important issue :

- if your function is copied in more than two files.

  > 💬 If you develop several workflows, you could rapidly get to tens of files that embed a copy of this function. Then, you would need to apply the modification to each copy of the function.

- if you simply forget to propagate the modifications to all the copies of your function, you will get a "versioning problem" where part of your workflow works with the current version of this function and another part of your workflow works with an old and deprecated version of this function : this is risky !

**⚙ How to avoid code duplication**

When a *helper function* is required more than once in your workflow, define this function in a module that shall be imported in the scripts of your worflow.

**⚙ Create and use a "utils" module**

1. Create one or more helper functions that could be used as a "util" in several scripts
2. Create a module `.py` file in the `Utils` folder and put the definition of helper function(s) in it
3. Use `import` keyword in a PYTHON script to import the module (as we did for any other library until now).

> ⚠ In step 3, the path of the module relative to the current script should be given (so the computer can find the module). To avoid this and to make the import easier and more generic, keep in mind that we added this line in the `/.bashrc` (for Unix) or `/.bash_profile` (for Mac) :
>
> ```
> export PYTHONPATH="${PYTHONPATH}:${TAF}/Utils"
> ```
>
> as described in Section Update PYTHONPATH.

We now propose exercises based on the example studied here (`test_read_file.py`) to apply the method defined above.

TAF course (*Preliminary*)- November 3, 2026

### I.3.b) Step 1 : create a helper function

**Ex. 3.1 | Creation of a helper function to read a `.root` file**

1. Enter the `${TAF}/Exercise` directory
2. Create a `TestReadFile` directory and enter it
3. Create a script named `test_read_file.py`, use the Visual Studio Code snippet (`py_tmpl_taf`) and fill the `main` function with the code presented in the method in Section Convert a TTree in pandas dataframe
4. Define a helper function called `get_df_from_rootfile` returning a `pandas` dataframe for a given `.root` file name, a given `TTree` and for a given set of branches.
5. Check that this function works as expected by calling it in the `main` function and printing the dataframe.

### I.3.c) Step 2 : create a module

**Ex. 3.2 | Create a module**

Inside the `Utils` folder :

1. create a module file named `taf_data_handler.py`
2. put the definition of the helper function `get_df_from_rootfile` added before in the `test_read_file.py`

⚠ Do not forget to `import` the necessary libraries inside this new module.

### I.3.d) Step 3 : call this module in a script

**Ex. 3.3 | Call the module in a script**

1. Enter the `${TAF}/Exercise/TestReadFile` directory
2. Make a copy of `test_read_file.py` named `test_read_file_with_utils.py`
3. In `test_read_file_with_utils.py`, remove the definition of `get_df_from_rootfile` and replace it with

```
from taf_data_handler.py import get_df_from_rootfile
```

Listing 3.5 – In the main function of `test_read_file_with_module.py`

4. Check that the execution of `test_read_file_with_utils.py` gives the same result as `test_read_file.py`

TAF course (*Preliminary*)- November 3, 2026

# II) *A parte* : make your script configurable

## II.1) Command line arguments

### The argparse

It is a PYTHON library used for command-line options, arguments and subcommands.

### Enable command line arguments for your PYTHON script

1. Import the library :

```
from argparse import ArgumentParser
```

> 💬 We only use the `ArgumentParse` method of the `argparse` library in this course.

2. Create an instance of `ArgumentParse` in the `if __name__ == "__main__"` block :

```
if __name__ == "__main__":
    parser = ArgumentParser(
        prog="program_name.py",
        description="Description of the program",
        epilog="Text at the bottom of help"
    )
```

And that is it! The next step is to add some arguments.

### Nota bene

You can access the details of your `argparse` implementation by running the `help` option :

```
$ python3 program_name.py -h
```

or

```
$ python3 program_name.py --help
```

### Add a switch argument

Let's assume we already created an object `parser` (see method above). We now need to use the `add_argument` method to add an argument.

```
parser.add_argument(
    "-f", "--flag", default=False, action="store_true", help="my flag"
)
```

Listing 3.6 – In the `if __name__ == "__main__"` block

where `f` is a short name for `flag`.

TAF course (*Preliminary*)- November 3, 2026

**Example – a debug switch**

To add a debug switch, on could use the following line :

```
parser.add_argument(
    "-d", "--debug", default=False, action="store_true", help="debug flag"
)
```

Listing 3.7 – In the `if __name__ == "__main__"` block

### Get the value of an argparse argument

To retrieve all the arguments, you just need to add the following line :

```
args = parser.parse_args()
```

Listing 3.8 – In the `if __name__ == "__main__"` block

Then you can access an argument through the `args` object.
To retrieve the value of the argument named `flag` :

```
flag = args.flag
```

Listing 3.9 – In the `if __name__ == "__main__"` block

**Example – a debug switch**

Coming back to the previous example. To get the value of the debug flag :

```
debug = args.debug
```

Listing 3.10 – In the `if __name__ == "__main__"` block

### Ex. 3.4 | Test the debug switch on a simple code

1. Enter the `${TAF}/Exercise` directory
2. Create a `TestCfg` directory and enter it
3. Create a new file named `test_debug_switch_argument.py`
4. Open this file with Visual Studio Code and type `py_tmpl_taf`
5. Adapt the code to replace the hardcoded `DEBUG` variable with a switch argument
6. Run the script with and without the `debug` option to test the difference.

TAF course (*Preliminary*)- November 3, 2026

## II.2) YAML configuration files

### YAML

YAML is a human-readable data serialisation language. A so-called YAML file has the extension `.yml`

It is structured in blocks via indentations.

> 💬 These indentation blocks can be translated into PYTHON dictionaries as we are about to see, making YAML a solid format for PYTHON configuration files.

> One advantage of the YAML file over the JSON file, is about comments. Indeed, comments are forbidden in a JSON file while authorised in YAML. This comes very handy to give some indications about the content of the file.

### II.2.a) Set the `argparse` functionality

### Add a YAML configuration file in argument

Let's assume we already created an object `parser`. To add an argument that will take then name of the configuration file :

```
parser.add_argument(
    "name_config_file",
    metavar="name_config_file",
    default="config.yml",
    help="YAML configuration file"
)
```

Listing 3.11 – In the `if __name__ == "__main__"` block

### Ex. 3.5 | Add a YAML configuration file in argument

1. Enter the `${TAF}/Exercise/TestCfg` directory
2. Make a copy of the file `test_debug_switch_argument.py` named `test_yaml_config.py`
3. Adapt the code of `test_yaml_config.py` to make it configurable (following the methods above)
4. Change the definition of the `main` function :

   ```
   def main(name_config_file: str, debug: bool):
   ```

   and adapt the code in the `if __name__ == "__main__"` block.
   *You need to retrieve the name of the config file and the value of the debug switch from `args`.*

TAF course (*Preliminary*)- November 3, 2026

### II.2.b) Read a YAML file in PYTHON

For this part, you need to have completed the section before (*i.e.* be able to get the name of the configuration file given in command line argument).

**From YAML to `dict`**

One of the great advantages of a YAML file is that its structure allows it to be easily converted into a PYTHON dictionary.

> 💬 This requires the `pyyaml` library, especially the `load` and `FullLoader` methods.

**Load YAML file as a `dict`**

1. Import the necessary tools from `pyyaml` library :

```python
from yaml import load as ymlLoad
from yaml import FullLoader as ymlFullLoader
```

Listing 3.12 – In the file preamble

> 💬 We give aliases to these tools (especially important for the `load` method that could also exist in another library).

2. Define an empty dictionary that will store the configuration :

```python
config: dict = {}
```

Listing 3.13 – In the `main` function

3. Open the configuration file :

```python
with open(name_config_file, "r", encoding="utf-8") as yml_config_file:
    # put code here
```

Listing 3.14 – In the `main` function

> ⚠ The variable `name_config_file` is the name of the configuration file obtained via `argparse` (see previous section).

4. Call the `pyyaml` methods to load the YAML file as a dictionary :

```python
config: dict = {}
name_config_file: str = "my_config.yml"
with open(name_config_file, "r", encoding="utf-8") as yml_config_file:
    config = ymlLoad(yml_config_file, ymlFullLoader)
```

Listing 3.15 – In the `main` function

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

It is a good habit to check that the YAML file import worked as expected. To do so, we can simply `print(config)`. As it is only a check, it is adapted to the `debug` flag :

```
if debug:
    LOG(config, "DEBUG")
```

Listing 3.16 – In the `main` function

⚠ This requires to add

```
from taf_logger import Logger as LOG
```

Listing 3.17 – In the preamble

See Section II.6).

However, when you want to `LOG` a dictionary, it is better to use `pformat` method from the `pprint` (Pretty Print) library as it proposes a nice layout and the possibility to define indentation as well as the width of the string in the terminal :

```
if debug:
    str_config = pformat(config, indent=2, width=100)
    LOG(f"\n{str_config}", "DEBUG")
```

Listing 3.18 – In the `main` function

💬 The return to the line `"\n"` is purely aesthetical.

⚠ This requires to add

```
from pprint import pformat
```

Listing 3.19 – In the preamble

**Ex. 3.6 | Quick test with a single-entry YAML file**

1. Create a single-entry YAML file named `simple_cfg.yml` :

```
key: value
```

Listing 3.20 – `simple_cfg.yml`

2. Use the method above to adapt `test_yaml_config.py` (from previous exercise) so that this script can convert the content of `simple_cfg.yml` into a `dict`
3. `LOG` the content of the obtained `dict`

**Use the `config` dictionary**

The `config` obtained in the previous method can be handled just like any other `dict` object.

**Example**

TAF course (*Preliminary*)- November 3, 2026

Here is an example of YAML file.

```yaml
info:
  experiment: CLINM
  campaign: CNAO April 2025
  beam:
    particle: 12C
    energy: 119.5 MeV/u
  runs: [1, 2, 3, 4, 5]

input:
  file: Data_CLINM_CNAO0425.root
  tree: tree
  branches: null
```

Listing 3.21 – `example_config.yml`

Let's assume that we already have a code that can read the YAML file and convert it into a dictionary (see previous exercise) named `config`.

**▶ Get first-level entries**

To get the configuration related to the `info` key in the YAML file, we just write :

```python
cfg_info = config["info"]
```

And to get the configuration related to the `input` key in the YAML file, we just write :

```python
cfg_input = config["input"]
```

**▶ Get second-level entries**

Then, to get second-level entries (or "sub-entries"), we follow the same procedure. For instance, if you want to access the value of the sub-key `experiment` in the key `info` :

```python
info_experiment = cfg_info["experiment"]
```

equivalent to

```python
info_experiment = config["info"]["experiment"]
```

**▶ Get lower-level entries**

You can keep following this procedure in the case of third-level entries, fourth-level entries, ... it depends on the structure and the depth of your YAML file.

**Ex. 3.7 | Train on an example of YAML file**

We use the file `example_config.yml` shown in the method above.

For every key, `LOG` the result of the corresponding entry.

1. `input`
2. `campaign`
3. `energy`
4. runs
5. branches

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

The keyword `null` in the YAML file is translated to `None` in PYTHON.

## II.3) Make file reading configurable

We are going to gather all the tools introduced in Sections Import and read a `.root` file and *A parte* : make your script configurable to write a *generic* PYTHON script that can read a `.root` file and retrieve a `TTree` out of it. The adjective *generic* is crucial here, meaning that once it will be written, we will not need to modify it : if we need to open another `.root` file or access another `TTree`, we will only need to create a new YAML configuration file!

**Make file-reading configurable**

1. Spot the configurables in the script and isolate them
2. Create a YAML file with the adapted structure

   > This step is really dependent on the context (*e.g.* here, it is a script to acces a `TTree` from a `rootfile`).

3. Adapt the script to read the configuration file from command line and get the configurables from the dictionary (obtained via `pyyaml` tools).

For the following exercises, we will use the script developed at the end of Section Import and read a `.root` file as a starting kit. Create a copy of this script in `${TAF}` directory and name it `read_rootfile.py`

**Ex. 3.8 | Make file-reading configurable**

Follow the method described above and apply it in `read_rootfile.py`

**Always provide a configuration example**

In your analysis workflow, the goal is to have PYTHON scripts as general as possible while applying the modifications to the configuration files only.
In the end, you would have one PYTHON script (for a given purpose) and several configuration files. Most of the time, these configuration files are personal (as they are analysis-dependent), so you do not provide them directly in your framework. If so, you need to provide a configuration file example as a starter toolkit for a newcomer.

## II.4) Add safeties

**Safeties in a configurable program**

To keep it simple, we will present two types of safeties here :

- the *non-executed*[a] safety : indicate comments in the YAML file. This is a good habit as it can provide a guide to the user;

TAF course (*Preliminary*)- November 3, 2026

▷ the *executed* safety : implement consistency checks in the code. This is the most robust option as it prevents the user from setting undesired configurations.

Of course, the best would be to do both.

**Nota bene**

Implementing consistency checks forces you to fully understand the limits of your code and could give you ideas on how to improve your code modularity or robustness.

a. Here, *executed* means that we address a piece of code that is actually executed (*e.g.* comments are not).

### Add comments in the YAML file

The syntax is identical to the one in PYTHON, you need to use the # symbol.

### Add consistency checks

1. You need to define limits for your configurables.
   - 💬 Most of the time, this addresses the variable type.
   - 💬 This is the most difficult part of the process as one could leave some important limit on the side of the road.
2. Define a helper function, typically named `check_config_consistency`, where the actual consistency checks will be performed.

**Example**

Let's assume one of the entries of the configuration has to be a `list` for the code to work properly. You have several possibilities :

▷ you can stop the program if the input does not match the right type :

```
if not isinstance(entry, list):
    LOG(f"The configurable {entry} must be a list, "FATAL")
```

▷ you can force the object to be a list :

```
if not isinstance(entry, list):
    entry = [entry]
```

### Ex. 3.9 | Add safeties to input file reading

1. Add comments to the configuration YAML file used in your own workflow.
2. Adapt the function defined to read an input file to make it safer.

TAF course (*Preliminary*)- November 3, 2026

# III) Analyse the data... and save the results

## III.1) Data Analysis

### III.1.a) Some pandas tips

In the following examples, we will manipulate a simplistic dataframe `df` such as :

```
>>> import pandas as pd
>>> d = {"A": [1, 2, 3, 4, 5]}
>>> df = pd.DataFrame(d)
>>> df
   A
0  1
1  2
2  3
3  4
4  5
```

Listing 3.22 – In the PYTHON console

**Add a column using `assign`**

There are several ways to add a column, but in the context of data analysis, we most often define a new column using information from other columns.

```
df["NameOfNewColumn"] = <some function of existing columns>
# OR
df_new = df.assign(NameOfNewColumn=<some function of existing columns>)
```

> 💬 The second method using `df.assign` performs a copy of the original dataframe which is not the case of the first method (`df[<NewColumnName>] = ...`). It is slower but better suiting for method chaining.

**Example**

Let's add a column `B` to `df` using a `lambda` function. This column will contain the square values of the column `A`.

```
>>> df
   A
0  1
1  2
2  3
3  4
4  5
>>> df["B"] = df["A"]**2
>>> df
   A   B
0  1   1
1  2   4
2  3   9
3  4  16
4  5  25
```

Listing 3.23 – In the PYTHON console

TAF course (*Preliminary*)- November 3, 2026

💬 To do the same with `assign`, one should do :

```
df = df.assign(B=lambda d: d["A"]**2)
```

Now, let's add a column `C` to `df` via a user-defined function. This column will contain the cube values of the column `A`.

```
>>> def cube(column: pd.Series):
...     return column**3
...
>>> df["C"] = cube(df["A"])
>>> df
   A   B    C
0  1   1    1
1  2   4    8
2  3   9   27
3  4  16   64
4  5  25  125
```

Listing 3.24 – In the PYTHON console

💬 To do the same with `assign`, one should do :

```
df = df.assign(C=cube(df["A"]))
```

## Apply a simple selection with query

You first need to define a selection on the dataframe. Such a selection is stored inside a `string` :

```
sel = "B > 18"
```

Listing 3.25 – Example of selection

Then, you need to call the `query` method of the `pandas.DataFrame` class, and use this selection :

```
df_sel = df.query(sel)
```

Listing 3.26 – Example of selection

You eventually get a new dataframe (called `df_sel` in the example above) that is a copy of the original dataframe `df` *but* only containing the rows that passed the selections (*i.e.* the rows where the value in the column `B` are strictly above 18).

⚠ `query` is rather slow, so it is counter-indicated to use it in `for` loops with numerous iterations, nor for binning (see next method for a faster approach).

### Example

Here is the result we get with our simplistic dataframe `df` :

```
>>> sel = "B > 18"
>>> df_sel = df.query(sel)
   A   B
4  5  25
```

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

Use `f-string` for the selection!

Indeed, very often, the cut value is a result of computations performed within the script : a very simple way to integrate such a value in a `string` is to use a `f-string`. The example above could be :

```
>>> my_lower_limit = 18
>>> sel = f"B > {my_lower_limit}"
>>> df_sel = df.query(sel)
   A   B
4  5  25
```

**Apply several selections**

The advantage of `query` is that you can put as many selections as you wish with a single call of this method.
The trick is to rely on a single `string` object for the selection. To do so, we use logical connectors :

- `"and"` for AND logic;
- `"or"` for OR logic.

**Example**

We have the following dataframe with 2 columns named `A` and `B` :

```
>>> df.query("B > 4 or A == 1")
   A   B
0  1   1
2  3   9
3  4  16
4  5  25
>>> df.query("B > 4 or A < 5")
   A   B
0  1   1
1  2   4
2  3   9
3  4  16
4  5  25
>>> df.query("B > 4 and A < 5")
   A   B
2  3   9
3  4  16
```

Listing 3.27 – In the PYTHON console

TAF course (*Preliminary*)- November 3, 2026

### The `inplace` option

A very important option, that you might encounter in several methods in `pandas`, is the `inplace` option. As you could expect it, this option is here to indicate if you wish that the modified[a] dataframe *replaces* the original one or not.

#### Example

For a given selection `sel`, if you wish :

- ▷ to replace the original dataframe with a new one, you could do :

```
df_sel = df.query(sel, inplace=False)
```

- ▷ *not* to replace the original dataframe (*i.e.* you wish to re-write it), you could do :

```
df.query(sel, inplace=True)
```

> 💬 By default, `inplace=False`

*a.* modified by the `pandas` method currently used

### Slice a dataframe into bins

#### Example

Let's suppose we defined a `numpy` array, called `binning`, with our desired binning of a quantity called `fZ`. In order to go from the original dataframe `df` to a sliced dataframe `df_sliced`, one could do :

```
df_sliced = (
    # apply cut on binning
    df.assign(Zbin=pd.cut(df["fZ"], bins=binning))
    # group by bins
    .groupby("Zbin", observed=True)
    # aggregate
    .agg(
        fEdepTot=("fEdep", "sum"),
        fEdepTotUnc=("fEdepUnc", lambda x: np.sqrt((x**2).sum())),
    )
)
```

> 💬 The columns present in `df_sliced` are the ones defined via the `agg` method. Here, we simply take the sum of all the $E_{\text{dep}}$ and perform the quadratic sum of related uncertainties within each bin.

> 💬 This is where the notations can become harder to follow : do not hesitate to refer to `pandas` documentation and forums to explore more sophisticated tools.

### III.1.b) Hands-on : energy deposits of photons and protons in a water volume

We are going to proceed to an *example* of data analysis through the following situation :

We consider a tank ($10 \times 10 \times 20$ cm$^3$) full of water, in cartesian frame $(x, y, z)$. We send a

TAF course (*Preliminary*)- November 3, 2026

beam of photons (later noted as $\gamma$ or `gamma`) at 8 MeV and propagating along $z$ towards the water tank. We measure the energy deposit ($E_{\text{dep}}$) of the photons at several points within the water tank (we have some 3D-matrix of $E_{\text{dep}}$ measures) with a very fine resolution[1]. Then, we do the same but for a beam of protons (later noted as $\mathrm{p}$ or `proton`) at 135 MeV, and also propagating along $z$ towards the water tank.
The objective is to analyse the data (values of $E_{\text{dep}}$ and associated uncertainties at several points in a 3D space) in order to represent the evolution of $E_{\text{dep}}$ as a function of $z$. Such a plot shows the BRAGG curve followed by the proton distribution (contrary to the photon one), highlighting the ballistic advantage of using charged particles to deposit energy in a more localised volume : this is one of the pillars of hadrontherapy (use ion beams for cancer treatments).

> 💬 Here we use the `pandas` library to manipulate data.

> 💬 In the following, the file `data.root` (available in the Seafile folder) will be referred to as "the data file".

### Ex. 3.10 | Analyse energy deposits of $\gamma$ and $\mathrm{p}$ in a water volume *Part 1*

1. Create a `${TAF}/Analysis` directory and enter it
2. Create a script named `analyse_edep.py`
3. Use all the tools explored until now to
   a. make this script configurable (with a YAML file containing an input configuration)
      > 💬 One could re-use the example of `example_config.yml` from Section YAML configuration files.
   b. convert the `TTree` in the data file into a `pandas` dataframe
4. Add a new column called `fEdepUnc` containing the *absolute* uncertainty on $E_{\text{dep}}$
5. Define a binning along the $z$ direction
   > 💬 Use a constant binning with a step of 1 or 2 mm for instance.
6. Create a dataframe dedicated to $\gamma$ rays, and another dataframe dedicated to protons
7. For each one of these two dataframes :
   a. Slice the dataframe into a $z$ bins
      > 💬 One could re-use the example code in the dedicated method presented in Section Some pandas tips.
   b. Add columns called `fZmin`, `fZmax`, `fZmean` containing the lower limit, upper limit and mean of a $z$ bin, respectively
   > 💬 It could help to define a helper function.
8. Add a new column containing the dose (in Gy).

1. The data used here come from simulation

TAF course (*Preliminary*)- November 3, 2026

> 💬 To go from $E_{\text{dep}}$ (in MeV) to dose (in Gy = J.m$^{-3}$), you need to convert the energy (MeV to J) and divide by the mass of each "water bin".

## III.2) Open and write a `.root` file with `uproot`

### III.2.a) Several writing options

#### ⚙ Open, write and close a file with `uproot`

To open and write a file named `name_ofile`, we can use different methods according to the objective :

- ▷ `recreate`[a] : allows to write content to the file while erasing its current content.
  > 💬 If the file does not exist, it is automatically created.
- ▷ `update` : allows to write content to the file without erasing its current content
- ▷ `create`[b] : allows to write content to a file only if this file does not exist yet.

```
name_ofile: str = "path_to_file.root"
with uproot.recreate(name_ofile) as file:
    # put code here
```

Listing 3.28 – In the main function of `test_write_file.py`

> 💬 As mentioned before, the `with` keyword allows for an automatic handling of the file closure (at the end of the `with` block).

*a.* the most commonly used.
*b.* rarely used.

### III.2.b) Write a dataframe as a `TTree`

#### ⚙ Write a `pandas.DataFrame` as a `TTree` in a `.root` file

> 💬 We assume that a `pandas.DataFrame` stored in a variable `df` is already defined, and possess three columns : `energy`, `momentum` and `id`

1. Build an empty `uproot.TTree` object by defining its name, the name of its branches as well as their type :

```
# define the name of the TTree
name_tree: str = "myTreeName"
# define the name and type of each branch
types_branch: dict = {"energy": "float", "momentum": "float", "id": "int"}
# create a uproot.TTree object
tree: uproot.TTree = file.mktree(name_tree, types_branch)
```

Listing 3.29 – Inside the `with` block

TAF course (*Preliminary*)- November 3, 2026

**Nota bene**

You can automatically retrieve the types from the dataframe :

```
types_branch: dict = df.dtypes.to_dict()
```

⚠ This should work just fine with simple types (*e.g.* `float`, `double`, `int`, `bool`, ...) but you should cross-check the content of `types_branch` (especially for more complex types as lists or arrays) :

```
if debug:
    LOG(types_branch, "DEBUG")
```

2. Fill this object with information contained in a `pandas.DataFrame`, using the `extend` method of the class `uproot.TTree` :

```
# define the name of the TTree
# fill the TTree
tree.extend(df.to_dict(orient="list"))
```

Listing 3.30 – Inside the `with` block

> 💬 The dataframe is converted into a `dict` object via the `pandas` method `to_dict`
>
> 💬 The `orient="list"` option is to get a dictionary structured this way : `dict like column -> [values]`

### III.2.c) Update `taf_data_handler.py` module

In Section The `Utils` folder, we created a module named `taf_data_handler.py` and fed it with a function providing a handle to `.root` file reading and conversion into a `pandas` dataframe. Now, we are going to add a function to this module to handle file writing.

**Ex. 3.11 | Creation of a helper function to write a `.root` file**

1. In `taf_data_handler.py`, define a helper function called `snapshot` recreating a `.root` file for a given `pandas` dataframe, name of `TTree` and branches
2. Call this function in your script :
   1. import the function of the module in your script
   2. calling it in the `main` function
3. Check that this function works as expected :
   - ▷ use the `get_df_from_rootfile` function to convert the `TTree` in the file you just wrote into a dataframe; then `LOG` the dataframe
   - ▷ 💬 If you have a `ROOT` installation, you can also explore the content of the written file with `TBrowser`
4. Once the helper function `snapshot` is well implemented and functional, you could make it more modular by letting the user choose between the `recreate`, `update` or `create` writing options

TAF course (*Preliminary*)- November 3, 2026

Once the helper function `snapshot` is well implemented and functional, you could make `snapshot` even more modular by letting the user choose between manual and automatic definition of `types_branch`

### III.2.d) Hands-on : energy deposits of photons and protons in a water volume

**Ex. 3.12 | Analyse energy deposits of $\gamma$ and $\mathrm{p}$ in a water volume *Part 2***

The Part 1 of this exercise is in the Section Hands-on : energy deposits of photons and protons in a water volume.

6. Save these two dataframes in ***separate*** `TTree` objects inside a ***same*** `.root` file

### III.2.e) Make file-writing configurable

**Ex. 3.13 | Make file-writing configurable**

Follow the method described Section Make file reading configurable and apply it in your analysis script.

This is a good opportunity to talk about output file name automatic definition.

**Automatic definition of output file name**

Whenever possible, it is a good habit to define the name of the output file automatically.

`f-string` might come in handy for this kind of manipulations.

One could allow the user to define its own output file name.

This is a good habit too. The best is to propose both options :

- ▷ an automatic definition
- ▷ a manual definition

of the output file name. This can easily be done via a configuration file.

**Example**

Let's suppose we have an input file named as follows :

```
name_infile: str = "Strasbourg_Experiment_18_07_2026.root"
```

We could add a prefix to indicate that the output file is a result of the analysis :

```
name_ofile: str = "AnalysisResults_" + name_infile
# or
name_ofile: str = f"AnalysisResults_{name_infile}"
```

TAF course (*Preliminary*)- November 3, 2026

**Ex. 3.14 | Configure output automatically**

Modify your script to allow automatic definition of the output file name as a function of the input file name.

# IV) Plots production

> 💬 This section is voluntarily short as it is very analysis-dependent (many many tools could be introduced but this is not the core of this course) : it is here to complete the analysis workflow.
> We will focus on the hands-on example treated in the previous section with a basic use of `matplotlib.pyplot`

The goal is to represent the $E_{\text{dep}}$ evolution as a function of $z$ (using the binning defined before), for both $\gamma$ and p.

**Separate analysing and plotting**

It is good practice to separate the analysis scripts from the "plotting" scripts.

> 💬 Indeed, if your analysis takes a non-negligible amount of time to run, it would be nonsense to re-run it only to produce a graph. It is way more advantageous to store the results of the analysis script in a file (*e.g.* a `.root` file) and to write a separate script that will produce plots from this result file.

> 💬 In many situations, creating a `Plots` (sub)folder containing "plotting" scripts as well as their output, is an easy way to apply this separation with pure analysis. It is also easier to find your way among your different results within your project.

**Ex. 3.15 | Representation of $E_{\text{dep}}$ as a function of $z$**

1. Create a `${TAF}/Plotting` directory and enter it
2. Create a file named `plot_edep_vs_z.py`
3. Open the output file of the analysis script and convert both `TTree` objects in `pandas` dataframes
4. Retrieve the $z$-binning and create a binning array
5. Represent the evolution of `fEdepTot` as a function of $z$

   > ⚠ Remember to always indicate axes labels and quantities units !

6. 🚀 Use `errorbar` method from `pyplot` to also display the uncertainties for each $z$ bin.

TAF course (*Preliminary*)- November 3, 2026